ПРОГРАММИРОВАНИЕ И ОБРАБОТКА ГРАФИЧЕСКОГО ИНТЕРФЕЙСА

Интерфейсы, делегаты, события, async/await

Лекция 3 — четыре инструмента, которые развязывают код

Фоменко М.Ю.

МАРШРУТ КУРСА

Где мы находимся

Было

Классы и объекты, пять принципов SOLID. Закончили на DIP: «зависеть от абстракций, а не от конкретики».

Сегодня

Что это за абстракции на практике. Интерфейс, делегат, событие и async/await — четыре разных инструмента.

Дальше

Тестирование: код, развязанный интерфейсами и событиями, наконец-то можно проверять автоматически.

Три из четырёх сегодняшних тем — это ответ на вопрос «как один класс общается с другим, не зная о нём».

КАРТА ЛЕКЦИИ

Четыре инструмента и четыре разных вопроса

Интерфейс

«С кем я работаю?»

Контракт: что объект обязан уметь. Реализацию можно подменить.

Делегат

«Что именно выполнить?»

Ссылка на метод, которую можно хранить в переменной и передавать.

Событие

«Кого оповестить?»

Объект сообщает, что у него что-то произошло, не зная, кто слушает.

async / await

«Ждать или не ждать?»

Долгая работа не блокирует поток — интерфейс остаётся живым.

Их часто путают между собой — поэтому разбираем строго по очереди.

1

Интерфейсы

Контракт: что класс обязан уметь — без единой строчки о том, как именно

ПРОБЛЕМА

Класс, прибитый гвоздями

Обработчик заказов сам создаёт себе логгер. Кажется удобным — ровно до первого изменения.

```
public class OrderProcessor
{
    private FileLogger _logger = new FileLogger();  // гвоздь

    public void Process(Order order)
    {
        _logger.Log("Обработка заказа " + order.Id);
    }
}
```

Логировать в консоль вместо файла — правим класс. Написать тест — некуда деться от записи на диск.

ОПРЕДЕЛЕНИЕ

Интерфейс

Интерфейс — это контракт: перечень методов и свойств, которые обязан предоставить реализующий его класс. Что уметь — есть, как именно — нет.

1 Только сигнатуры

Имена, параметры, типы возврата. Тел методов нет (почти — см. сноску).

2 Нельзя создать

new ILogger() не компилируется. Создаётся всегда конкретный класс.

3 Можно много

Класс наследует один базовый класс, но реализует сколько угодно интерфейсов.

Начиная с C# 8 интерфейс может иметь реализацию по умолчанию — приём редкий, для развития чужих библиотек.

КОД

Объявление и реализация

```
public interface ILogger
{
    void Log(string message);      // ни тела, ни модификатора
    bool IsEnabled { get; }        // свойство тоже можно
}

public class FileLogger : ILogger
{
    public bool IsEnabled => true;   // public обязателен

    public void Log(string message)
        => File.AppendAllText("log.txt", message);
}
```

Забыли реализовать хоть один член — код не соберётся. Компилятор следит за соблюдением контракта.

код

Переменная типа интерфейса

Тип переменной — контракт. Что за ним стоит, вызывающий код знать не обязан.

```
ILogger logger = new FileLogger();      // сегодня файл
logger = new ConsoleLogger();           // завтра консоль
logger.Log("Заказ принят");             // вызов не изменился

var loggers = new List<ILogger>         // и то, и другое сразу
{
    new FileLogger(),
    new ConsoleLogger()
};

foreach (var l in loggers) l.Log("Старт");
```

ДО / ПОСЛЕ

Зависимость приходит снаружи

Было: класс решает сам

```
class OrderProcessor
{
    private FileLogger _log
        = new FileLogger();

    // класс знает,
    // куда пишется лог
}
```

Стало: решает вызывающий

```
class OrderProcessor
{
    private readonly ILogger _log;

    public OrderProcessor(ILogger l)
    {
        _log = l;     // приходит извне
    }
}
```

Это и есть буква D из SOLID в готовом виде. Приём называется внедрением зависимости через конструктор.

КОД

Зачем это на практике: подмена

Раз тип — контракт, вместо настоящей реализации можно подсунуть любую другую.

```
class FakeLogger : ILogger        // ничего никуда не пишет
{
    public List<string> Messages = new();
    public bool IsEnabled => true;
    public void Log(string m) => Messages.Add(m);
}

var fake = new FakeLogger();
var processor = new OrderProcessor(fake);
processor.Process(order);

// теперь можно проверить, что именно он записал
```

КОД

Несколько контрактов сразу

```
interface IReadable  { string Read(); }
interface IWritable  { void Write(string data); }

// интерфейс может наследовать другой интерфейс
interface IStorage : IReadable, IWritable
{
    void Clear();
}

// класс реализует сколько нужно
class FileStorage : IStorage { /* три метода */ }
class ReadOnlyCache : IReadable { /* только Read */ }
```

Тот самый ISP: два маленьких контракта лучше одного большого, потому что кэшу незачем уметь писать.

СРАВНЕНИЕ

Абстрактный класс или интерфейс

	абстрактный класс	интерфейс
Реализация методов	может быть	нет (кроме default)
Поля, состояние	есть	нет
Сколько на класс	один	сколько угодно
Смысл связи	«является» (Circle — Shape)	«умеет» (Button — IClickable)
Когда брать	есть общий код для родни	нужен контракт между слоями

Простое правило: сомневаетесь — берите интерфейс. Наследование связывает жёстче, чем кажется.

ИТОГ ЧАСТИ 1

Интерфейсы: что забираем с собой

1. Интерфейс — контракт. Он говорит, что объект умеет, и молчит о том, как.
2. Переменная типа интерфейса позволяет подменить реализацию, не трогая вызывающий код.
3. Зависимость лучше принимать в конструкторе, а не создавать внутри класса.
4. Интерфейс — это шов, по которому код разбирается на части: для тестов, для замены, для роста.

Но интерфейс отвечает только на вопрос «с кем работать». Он не умеет передать одно конкретное действие.

2

Делегаты

Метод, который можно положить в переменную и передать дальше

ПРОБЛЕМА

Три почти одинаковых метода

Отличаются они ровно одной строчкой — условием отбора.

```
List<Order> FindExpensive(List<Order> all)
    => all.Where(o => o.Total > 1000).ToList();

List<Order> FindToday(List<Order> all)
    => all.Where(o => o.Date == DateTime.Today).ToList();

List<Order> FindCancelled(List<Order> all)
    => all.Where(o => o.IsCancelled).ToList();

// завтра попросят четвёртый вариант. И пятый.
```

Передать данные в метод умеем давно. Сегодня учимся передавать в метод само действие.

ОПРЕДЕЛЕНИЕ

Делегат

Делегат — это тип, объект которого хранит ссылку на метод. Ссылку можно положить в переменную, передать в другой метод и вызвать позже.

Что важно

Делегат описывает сигнатуру: типы параметров и тип возврата. Подойдёт любой метод, который в неё укладывается.

Аналогия

Номер телефона мастера. У вас не сам мастер, а способ его позвать. Кого именно вы записали — ваше дело.

Имя метода без скобок — это не вызов, а ссылка на него. Скобки означают «выполнить прямо сейчас».

КОД

Объявление и вызов

```
// 1. описали сигнатуру
public delegate int Operation(int a, int b);

static int Add(int a, int b) => a + b;
static int Multiply(int a, int b) => a * b;

// 2. положили метод в переменную — БЕЗ скобок
Operation op = Add;

// 3. вызвали через переменную
Console.WriteLine(op(2, 3));          // 5
Console.WriteLine(op.Invoke(2, 3));   // то же самое

op = Multiply;                        // переключили
Console.WriteLine(op(2, 3));          // 6
```

КОД

Делегат как параметр метода

Тот самый способ передать действие — и убрать дублирование с прошлого слайда.

```
delegate bool Check(Order order);

List<Order> Find(List<Order> all, Check condition)
{
    var result = new List<Order>();
    foreach (var o in all)
        if (condition(o)) result.Add(o);   // чужое условие
    return result;
}

var expensive = Find(orders, o => o.Total > 1000);
var cancelled = Find(orders, o => o.IsCancelled);
```

код

Один делегат — несколько методов

```
Notify notify = SendEmail;
notify += SendSms;          // теперь в списке два метода
notify += WriteToLog;       // три

notify("Заказ готов");      // вызовутся все три, по порядку

notify -= SendSms;          // отписали один
notify("Заказ доставлен"); // вызовутся два
```

Если у делегата есть возвращаемое значение, вы получите результат только последнего метода.
Поэтому многоадресные делегаты почти всегда объявляют с void.

ГОТОВЫЕ ТИПЫ

Свой delegate писать чаще всего не нужно

Тип	Что описывает	Пример
Action	Ничего не возвращает	Action<string> log = Console.WriteLine;
Func	Возвращает значение	Func<int, int, int> add = (a, b) => a + b;
Predicate	Возвращает bool	Predicate<Order> big = o => o.Total > 1000;

В реальном коде свой delegate объявляют редко — Action и Func закрывают почти всё.

У Func последний параметр в угловых скобках — это тип возвращаемого значения, а не ещё один аргумент.

КОД

Лямбда-выражения

Если метод нужен один раз и только здесь — заводить его отдельно незачем.

```
// было: отдельный метод ради одной строки
static bool IsBig(Order o) { return o.Total > 1000; }
var big = Find(orders, IsBig);

// стало: лямбда прямо на месте
var big2 = Find(orders, o => o.Total > 1000);

// несколько параметров и тело из нескольких строк
Func<int, int, int> max = (a, b) =>
{
    if (a > b) return a;
    return b;
};
```

ИТОГ ЧАСТИ 2

Делегаты: что забираем с собой

1. Делегат — переменная, в которой лежит метод. Имя без скобок — ссылка, со скобками — вызов.
2. Главное применение: передать в чужой метод своё поведение, не переписывая чужой метод.
3. Через += можно подписать несколько методов, через −= отписать.
4. Свой delegate объявляют редко: обычно хватает Action, Func и Predicate.

И тут возникает проблема: если делегат лежит в публичном поле, любой чужой код может его стереть или вызвать.

3

События

Объект сообщает, что у него что-то произошло, и не знает, кто это слушает

ПРОБЛЕМА

Что не так с публичным делегатом

```
public class OrderService
{
    public Notify OnOrderReady;      // публичное поле-делегат
}

// а теперь — из любого чужого класса:
service.OnOrderReady = null;         // стёрли ВСЕ подписки
service.OnOrderReady("Готово!");     // позвали от чужого имени
service.OnOrderReady = MyHandler;    // = вместо += , затёрли
```

Механизм подписки нужен, а вот право всё сломать — нет. Ровно эту границу и проводит ключевое слово event.

ОПРЕДЕЛЕНИЕ

Событие

Событие — это делегат, обёрнутый ограничениями. Снаружи с ним можно делать только две вещи.

Снаружи можно

Подписаться: += обработчик
Отписаться: −= обработчик

Снаружи нельзя

Присвоить целиком (=), обнулить, и главное — вызвать. Инициировать событие может только класс-владелец.

Издатель решает, когда событие произошло. Подписчики решают, реагировать им или нет.

КОД

Объявление, генерация, подписка

```
public class OrderService
{
    public event Action<string> OrderReady;   // событие

    public void Complete(Order order)
    {
        // ... бизнес-логика ...
        OrderReady?.Invoke("Заказ " + order.Id);  // генерируем
    }
}

var service = new OrderService();
service.OrderReady += text => Console.WriteLine(text);
service.OrderReady += text => File.AppendAllText("log.txt", text);
```

ПОДВОДНЫЙ КАМЕНЬ

Почему именно ?.Invoke

Так падает

```
OrderReady("Готово");

// если никто не подписался,
// событие равно null
// → NullReferenceException
```

Так правильно

```
OrderReady?.Invoke("Готово");

// нет подписчиков —
// вызов просто не произойдёт,
// программа живёт дальше
```

И обратная сторона: подписались через += — не забудьте отписаться через −=.

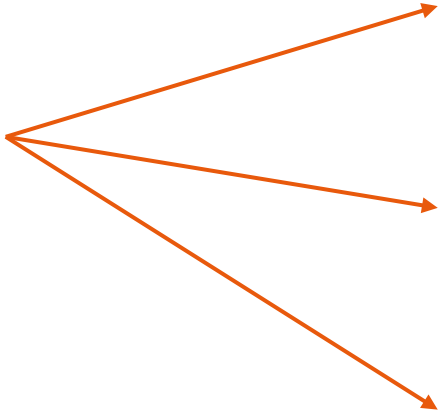
Живая подписка держит объект в памяти: подписчик не соберётся сборщиком мусора, пока издатель жив.

КАК ЭТО РАБОТАЕТ

Издатель и подписчики

Издатель

OrderService
событие OrderReady



Логгер

пишет в файл

Уведомления

шлёт письмо клиенту

Статистика

считает заказы за день

Издатель не знает подписчиков по имени. Добавить четвёртого — написать класс и подписать его, издатель не меняется.

КОГДА ЧТО

Делегат или событие

Делегат

- Нужно передать метод как аргумент
- Нужно сохранить метод и вызвать позже
- Вызывает тот, кто его получил
- Пример: условие отбора, стратегия расчёта

Событие

- Нужно оповестить о том, что произошло
- Подписчиков может быть ноль или много
- Вызывает только владелец события
- Пример: заказ готов, кнопка нажата, файл сохранён

Делегат — «сделай вот это». Событие — «имейте в виду, у меня вот что случилось».

4

async / await

Долгая работа не должна превращать программу в кирпич

ПРОБЛЕМА

У интерфейса есть только один поток

Dispatcher Thread

В WPF есть ровно один главный поток, и только он имеет право менять элементы интерфейса. Он же перерисовывает окно и обрабатывает клики.

Что происходит

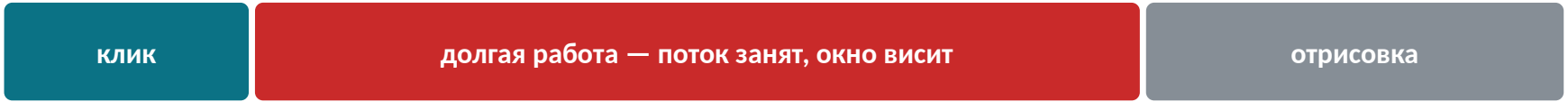
Запустили долгую работу прямо в обработчике кнопки — поток занят ею целиком. Окно не перерисовывается, не отвечает на клики: «Не отвечает».

Пользователь не знает, думает программа или умерла. Для него это просто сломанное приложение.

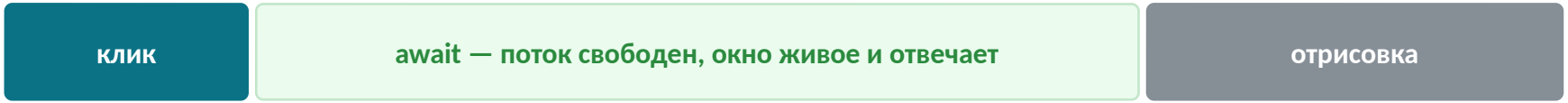
ДВЕ ШКАЛЫ ВРЕМЕНИ

Что меняет await

Синхронно



Асинхронно



работа идёт где-то ещё: диск, сеть, другой поток

await не ускоряет работу. Он освобождает поток на время ожидания — и этого достаточно, чтобы окно осталось живым.

КОД

Task, async, await

```
// было: поток стоит и ждёт
private void Button_Click(object sender, RoutedEventArgs e)
{
    var data = LoadData();      // 20 секунд паралича
    Show(data);
}

// стало: поток отпущен на время ожидания
private async void Button_Click(object sender, RoutedEventArgs e)
{
    var data = await LoadDataAsync();   // ждём, не блокируя
    Show(data);                         // выполнится после
}

async Task<string> LoadDataAsync() { /* ... */ }
```

КАК ЭТО УСТРОЕНО

Три факта, которые снимают половину вопросов

1. await не создаёт поток. Он говорит: «эта работа займёт время, верни поток тому, кто его ждёт».

2. Метод с async возвращает Task — обещание результата, а не сам результат. Task<T> — обещание значения типа T.

3. Код после await продолжится сам, когда задача завершится, и в WPF вернётся в поток интерфейса.

Если внутри async-метода нет ни одного await — компилятор предупредит: метод выполнится полностью синхронно.

ДВЕ РАЗНЫЕ СИТУАЦИИ

Ждём или считаем

Ждём чужой ответ (I/O)

Сеть, файл, база данных. Процессор в это время не занят — просто ждём.

Считаем сами (CPU)

Обработка изображения, расчёт, разбор большого файла. Процессор загружен работой.

```
var text = await httpClient.GetStringAsync(url);   // ждём — await

var result = await Task.Run(() => HeavyCalc(data)); // считаем — Task.Run
```

Task.Run уводит вычисление в поток из пула, await дожидается результата, не блокируя интерфейс.

ЧТОБЫ НЕ НАСТУПИТЬ

Три грабли

async void

Допустим только для обработчиков событий. В остальных случаях — async Task, иначе исключение внутри метода не поймать.

.Result и .Wait()

Блокируют поток и в приложении с UI приводят к дедлоку: поток ждёт задачу, а задача ждёт поток.

Асинхронность до конца

Если метод стал async, вызывающий тоже должен стать async. Обрывать цепочку посередине — источник странных зависаний.

СКВОЗНОЙ ПРИМЕР

Акт 1: как делать не надо

Кнопка «Запустить», долгая обработка и прогресс-бар. Пишем в лоб.

```
class DataProcessor
{
    public ProgressBar Bar;          // процессор знает про WPF

    public void Run()
    {
        for (int i = 0; i <= 100; i++)
        {
            Thread.Sleep(100);       // «работа»
            Bar.Value = i;           // и сам рисует
        }
    }
}
```

Две беды сразу: логика прибита к конкретному элементу интерфейса, и поток UI заблокирован — прогресс не виден.

СКВОЗНОЙ ПРИМЕР

Акт 2: интерфейс развязывает логику и UI

```
interface IProgressReporter { void Report(int percent); }

class DataProcessor
{
    private readonly IProgressReporter _reporter;
    public DataProcessor(IProgressReporter r) { _reporter = r; }

    public void Run()
    {
        for (int i = 0; i <= 100; i++)
        {
            Thread.Sleep(100);
            _reporter.Report(i);     // про WPF больше не знает
        }
    }
}
```

Архитектурно стало хорошо: процессор тестируем и не зависит от UI. Но окно всё ещё висит — интерфейс не про время.

СКВОЗНОЙ ПРИМЕР

Акт 3: событие плюс await

```
class DataProcessor
{
    public event Action<int> ProgressChanged;   // кричим в пустоту

    public async Task RunAsync()
    {
        for (int i = 0; i <= 100; i++)
        {
            await Task.Delay(100);              // не блокируем
            ProgressChanged?.Invoke(i);
        }
    }
}

processor.ProgressChanged += p => bar.Value = p;   // рисует окно
processor.ProgressChanged += p => log.Add(p);      // пишет лог
await processor.RunAsync();
```

Процессор не знает ни про прогресс-бар, ни про лог, ни сколько у него слушателей — и при этом окно живое.

ШПАРГАЛКА

Четыре инструмента

Инструмент	Решает задачу	Ключевые слова
Интерфейс	Подменяемая реализация, контракт между слоями	interface, : ILogger
Делегат	Передать действие как аргумент	delegate, Action, Func
Событие	Оповестить неизвестных заранее подписчиков	event, +=, ?.Invoke
async / await	Не блокировать поток на время ожидания	async, await, Task

Общая мысль: каждый из четырёх убирает лишнее знание одного класса о другом. Именно это и называется развязкой.

К СЛЕДУЮЩЕМУ ЗАНЯТИЮ

Что дальше

Лекция 4	Тестирование. Всё, что мы сегодня развязали, наконец можно проверить автоматически — заглушка вместо настоящей зависимости больше не фокус, а рабочий приём.
Проверить себя	Найти в своём коде место, где класс создаёт зависимость внутри себя через new, и переписать через интерфейс в конструкторе.
Поиграться	Взять любой свой обработчик кнопки с долгой операцией и перевести его на async/await. Разница видна сразу.

Вопросы

Интерфейсы, делегаты, события, async/await